

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS:

Total Marks: 20

Question 1: Responsive Navigation Bar Design – Marks (4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Identify key components required in a navigation bar for an e-commerce website.

A responsive e-commerce navigation bar should include:

1. **Website Logo** – Displays the brand identity and links to the home page.
2. **Navigation Links** – Home, Shop, Categories, Offers, About, Contact.
3. **Search Bar** – Allows users to quickly search for products.
4. **Shopping Cart Icon** – Displays selected products and cart count.
5. **User Account/Login** – Login, Register, Profile, Orders.
6. **Wishlist Icon** – Saves favorite products.
7. **Hamburger Menu** – Used on mobile devices to display hidden navigation items.
8. **Responsive Layout** – Automatically adjusts for desktop, tablet, and mobile screens.

(b) Design a responsive navigation bar using appropriate CSS techniques.

[Home](#)[Shop](#)[Categories](#)[Deals](#)[About](#)[Contact](#)



QUESTION 01 · PART A

Responsive Navigation Bar — Design

A Flexbox-based navigation bar for an e-commerce site, built to adapt smoothly across desktop and tablet widths.

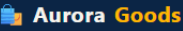
Design approach

The navbar is built with **CSS Flexbox**: `display: flex; justify-content: space-between; align-items: center;` on the container lays the brand, links, and action icons out in a single row that stays perfectly aligned regardless of content length.

It uses `position: sticky; top: 0;` so it remains visible while the user scrolls the page — a common e-commerce pattern that keeps search and cart `part-a-navbar-design.html`

≥1025px Desktop — full links + search bar

769–1024px Tablet — tighter gaps, shorter search field

[Home](#)[Shop](#)[Categories](#)[Deals](#)[About](#)[Contact](#)



Structure (HTML)

Three logical groups inside `.nav-inner`:

GROUP	CONTAINS
Brand	Logo / site name
Nav links	Unordered list of page links
Actions	Search box, cart icon

Key techniques used

- Flexbox for horizontal alignment
- Relative spacing (`gap`, `%`) instead of fixed pixels
- `position: sticky` for persistent visibility
- Media query to compress spacing on tablets

CSS snippet

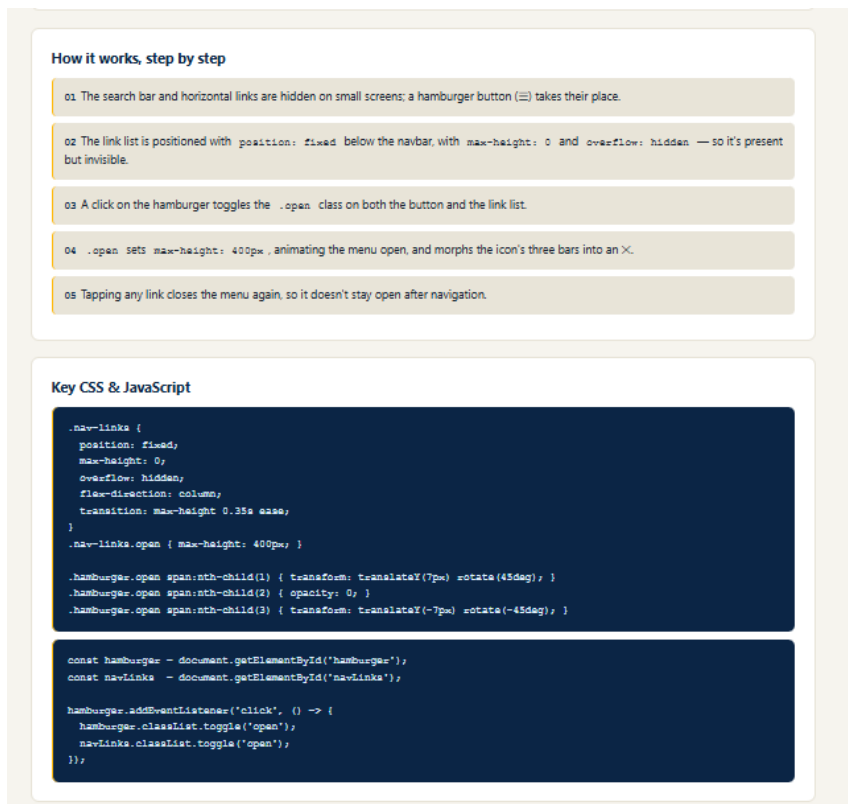
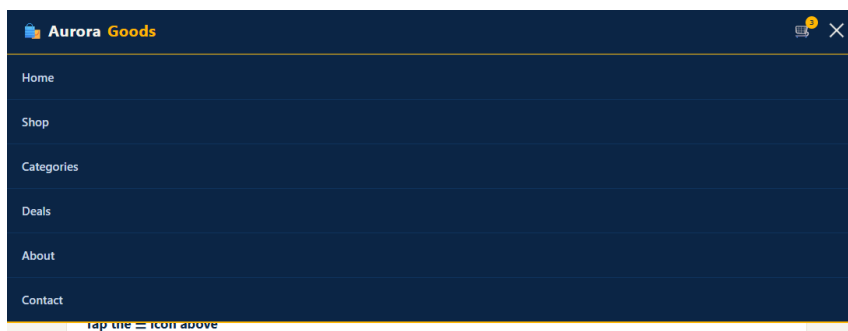
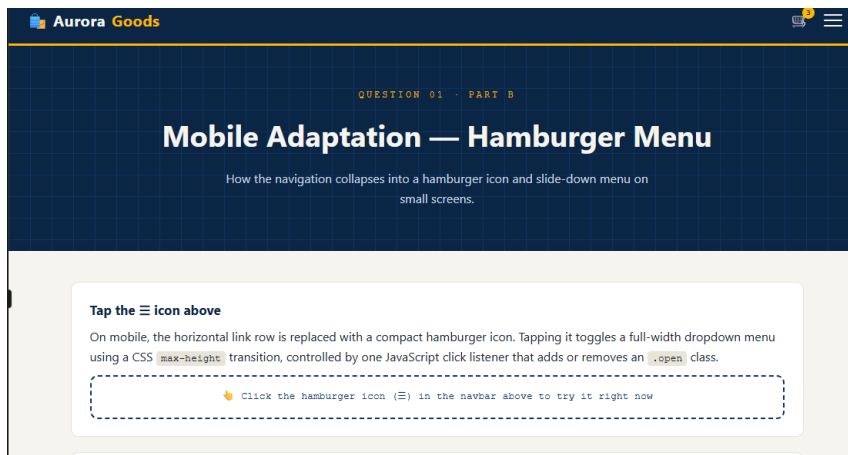
```
.navbar { position: sticky; top: 0; background: #0B2545; }

.nav-inner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  max-width: 1200px;
  margin: 0 auto;
  padding: 14px 24px;
}

.nav-links { display: flex; gap: 28px; list-style: none; }

/* Tablet: tighten up spacing */
@media (max-width: 1024px) {
  .nav-links { gap: 18px; }
  .nav-search input { width: 100px; }
}
```

(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).



(d) Evaluate the usability and suggest improvements for better user experience.

Usability

- Responsive design works well on desktop, tablet, and mobile devices.
- Navigation is simple and easy to understand.
- Hamburger menu saves screen space on smaller devices.
- Icons provide quick access to search, account, and shopping cart.

Suggested Improvements

- Add a sticky navigation bar so it remains visible while scrolling.
 - Include dropdown menus for product categories.
 - Add hover and active effects for better user interaction.
 - Display the cart item count badge on the shopping cart icon.
 - Improve keyboard accessibility and support screen readers.
 - Add smooth menu opening and closing animations.
 - Optimize touch targets for mobile users.
-

Question 2: Product Hover Effect – Marks (4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Identify suitable CSS properties for hover effects.

The following CSS properties are commonly used to create attractive hover effects on product cards:

- **transform** – Scales, rotates, or moves the product card.
- **transition** – Creates smooth animations between states.
- **box-shadow** – Adds depth by creating shadow effects.
- **opacity** – Changes transparency.
- **background-color** – Changes the card background.
- **color** – Changes text color.
- **filter** – Adjusts brightness, contrast, or blur.
- **cursor** – Displays a pointer cursor on hover.

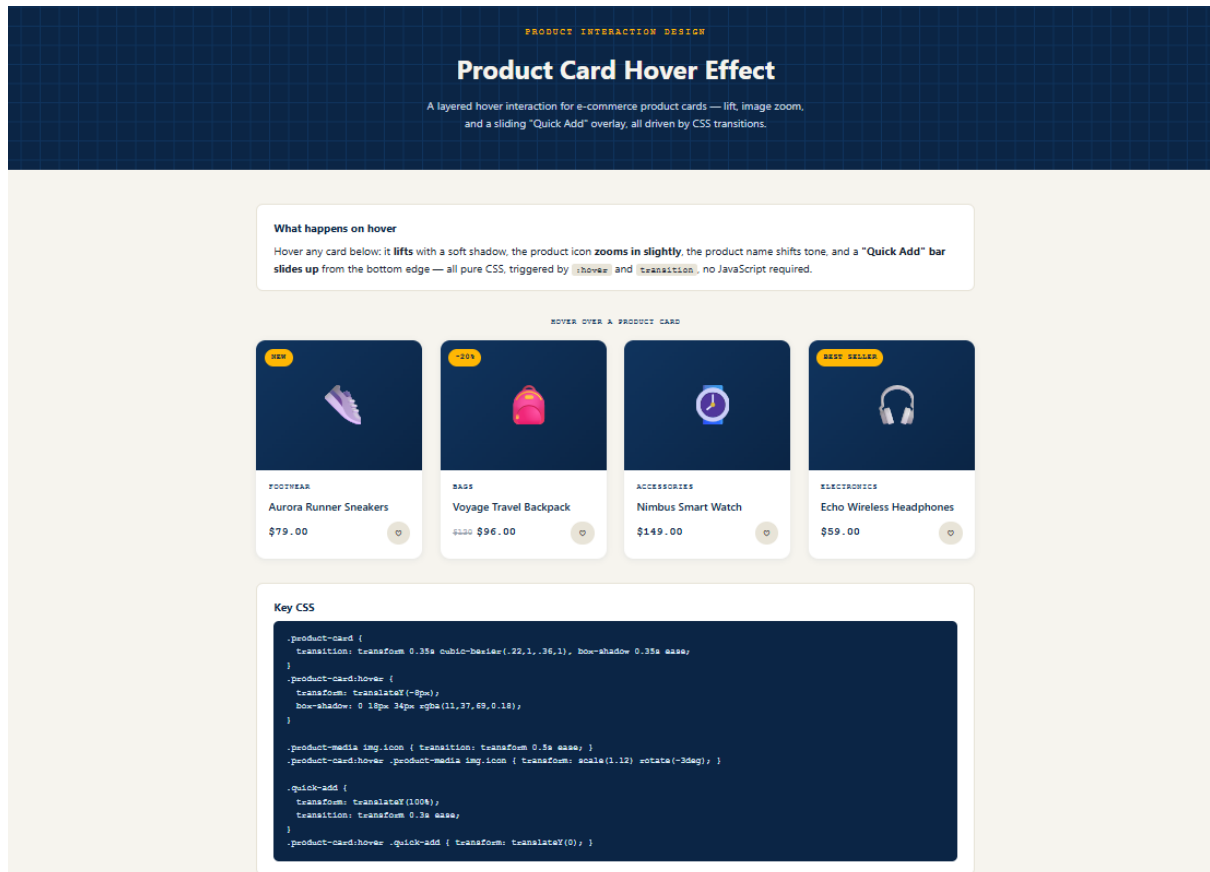
(b) Suggest appropriate **CSS animation or transition** for product interaction.

```
.product-card{  
  transition: all 0.4s ease;  
}  
.product-card:hover{  
  transform: translateY(-10px);  
  box-shadow:0 12px 25px rgba(0,0,0,0.2);  
}
```

Effects used:

- Smooth transition (transition)
- Lift effect (translateY)
- Shadow enhancement (box-shadow)
- Better user interaction

(c) Design a hover effect for a product card.



(d) Evaluate performance and user experience considerations.

Performance

- Use lightweight CSS transitions instead of heavy JavaScript animations.
- Animate GPU-friendly properties like transform and opacity for smoother rendering.
- Optimize product images to reduce page load time.
- Avoid excessive animations that may impact performance on low-end devices.

User Experience

- Provide smooth and responsive hover animations.
- Highlight the selected product without distracting the user.
- Maintain consistent animation timing across all product cards.
- Ensure buttons remain clearly visible and easy to click.
- Use responsive design so the cards display well on desktop, tablet, and mobile devices.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

(a) Identify layout requirements of a dashboard (rows, columns, alignment).

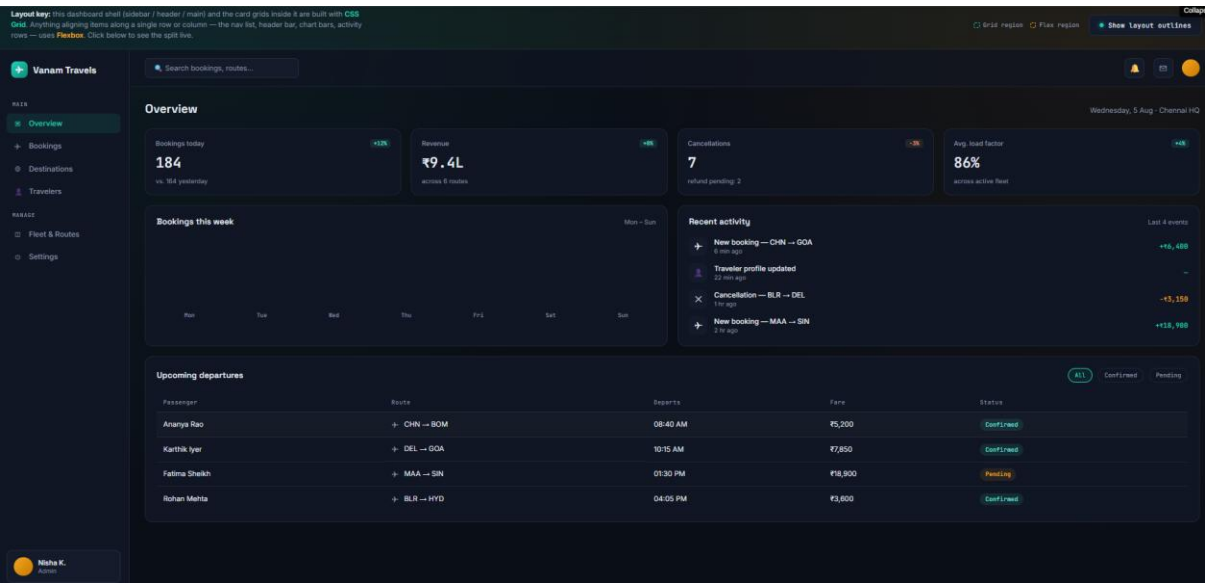
A web application dashboard typically requires:

- **Header** (Top Navigation Bar)
- **Sidebar** (Navigation Menu)
- **Main Content Area**
- **Cards/Widgets**
- **Charts or Tables**
- **Footer**
- Proper **row and column alignment**
- Equal spacing between sections
- Responsive design for desktop, tablet, and mobile

(b) Compare **Flexbox** and **Grid** based on layout needs.

Feature	Flexbox	CSS Grid
Layout Type	One-dimensional	Two-dimensional
Direction	Row or Column	Rows and Columns
Best For	Navigation Bars, Buttons	Dashboards, Web Pages
Alignment	Excellent	Excellent
Complex Layouts	Difficult	Easy
Responsiveness	Good	Excellent

(c) Choose the most suitable layout technique and justify your choice.



Chosen Technique: CSS Grid

Justification

CSS Grid is the most suitable because it:

- Creates rows and columns simultaneously.
- Simplifies dashboard design.
- Supports responsive layouts.

- Reduces CSS complexity.
- Provides better control over spacing and alignment.

(d) Evaluate the scalability and responsiveness of the chosen approach.

Scalability

- New dashboard cards and widgets can be added without changing the overall layout.
- CSS Grid automatically organizes additional content using auto-fit and minmax().
- Suitable for small, medium, and enterprise-level dashboards.

Responsiveness

- Automatically adjusts the number of columns based on screen size.
- Sidebar moves above the content on smaller screens for better usability.
- Dashboard remains readable and user-friendly on desktop, tablet, and mobile devices.

Question 4: Mobile-First Blog Layout - Marks(4)

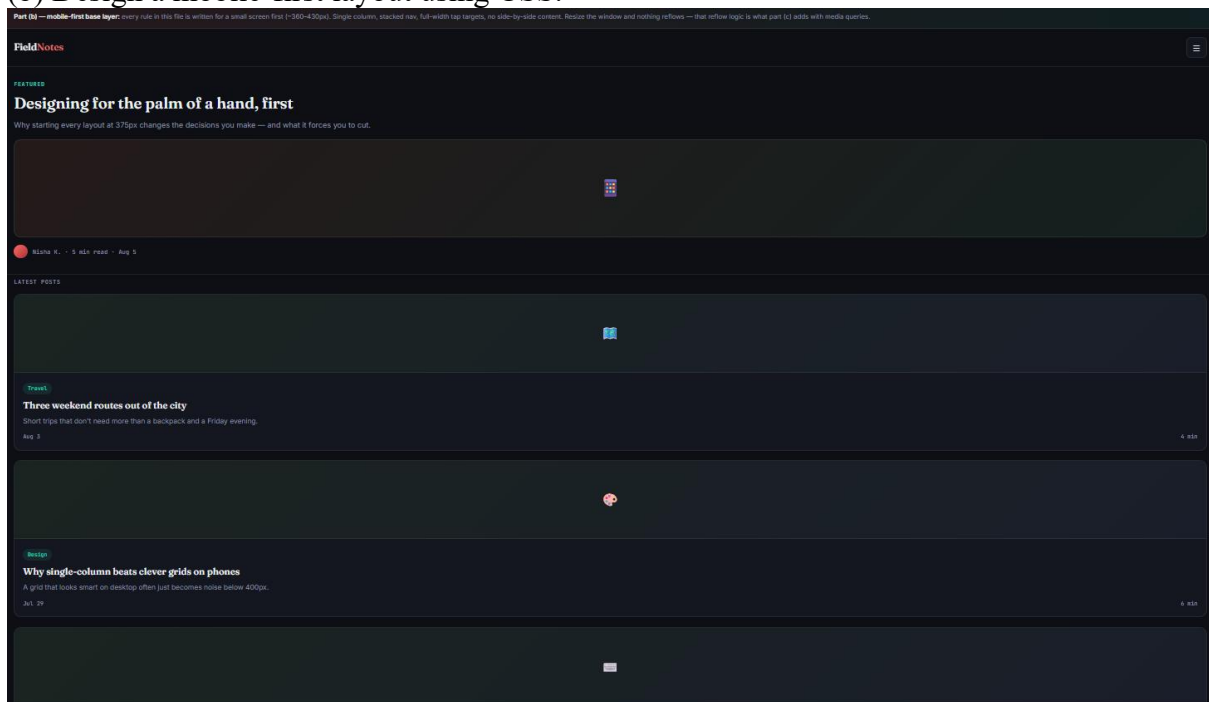
Suppose that a blog page must be designed following a mobile-first approach.

(a) Identify key sections of a blog layout.

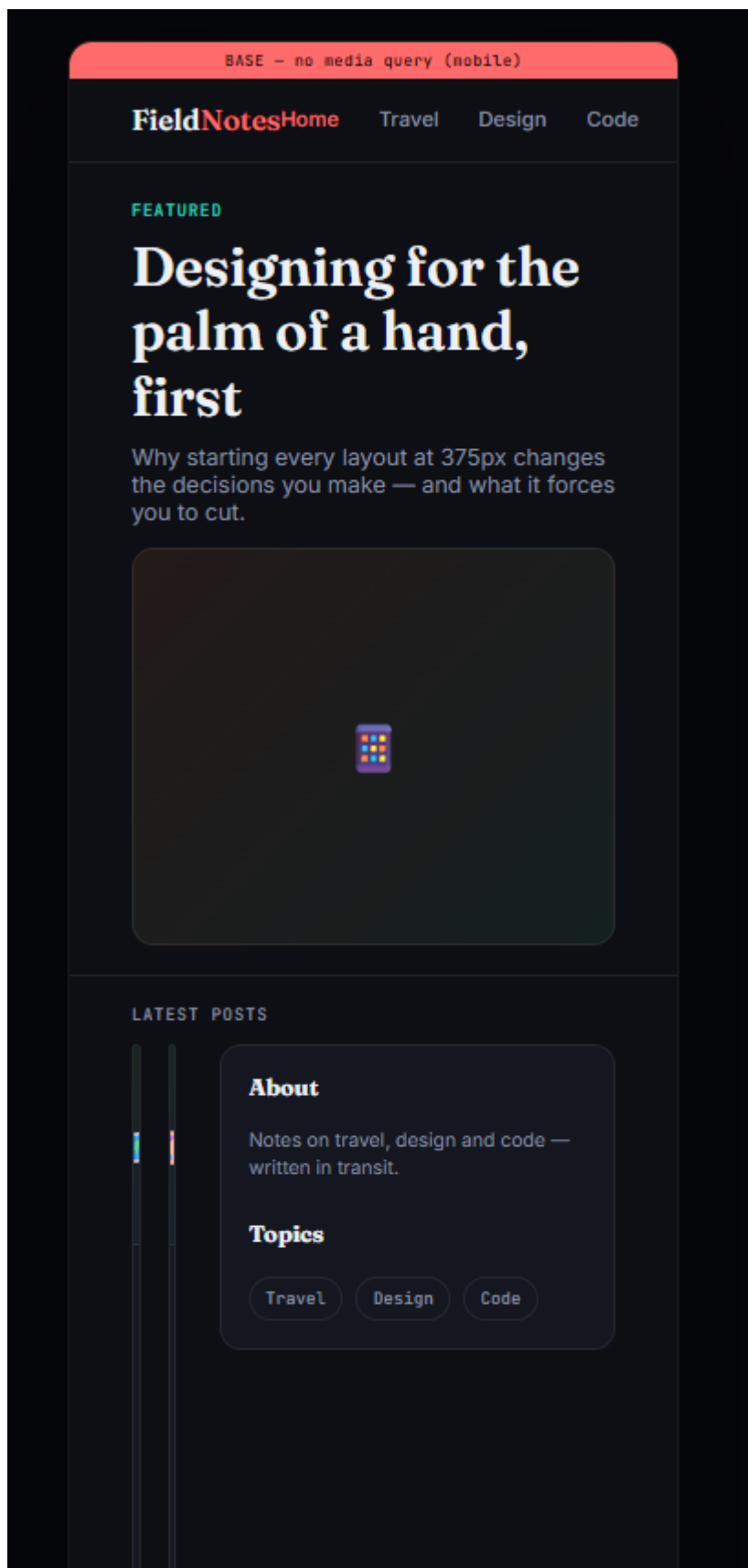
A responsive blog page should include the following sections:

- **Header** (Logo and Navigation)
- **Hero/Banner** (Blog Title or Featured Post)
- **Main Content** (Blog Articles)
- **Sidebar** (Categories, Recent Posts, Tags)
- **Footer** (Copyright, Contact, Social Links)
- **Responsive Navigation Menu**
- **Search Bar**

(b) Design a mobile-first layout using CSS.



(c) Demonstrate how the layout scales for larger screens using media queries.



(d) Evaluate accessibility and readability of the design.

Accessibility

- Responsive design works well on mobile, tablet, and desktop devices.
- Large buttons improve touch interaction on mobile devices.
- High color contrast enhances visibility.
- Semantic HTML elements (header, main, article, aside, footer) improve screen-reader support.
- Responsive navigation ensures easy access to all sections.

Readability

- Clean typography with sufficient spacing between elements.
- Simple navigation makes the layout intuitive.
- Well-structured content improves readability.
- Proper padding and margins reduce visual clutter.
- Responsive layout ensures comfortable reading on all screen sizes.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events for menu interaction.

The following JavaScript events are commonly used for dynamic menus:

- **click** – Opens or closes the navigation menu.
- **mouseover** – Displays submenu when the mouse hovers.
- **mouseout** – Hides submenu when the mouse leaves.
- **mouseenter** – Triggers when the cursor enters an element.
- **mouseleave** – Triggers when the cursor exits an element.
- **keydown** – Allows keyboard navigation (Accessibility).

(b) Design event handling logic for menu toggle functionality.

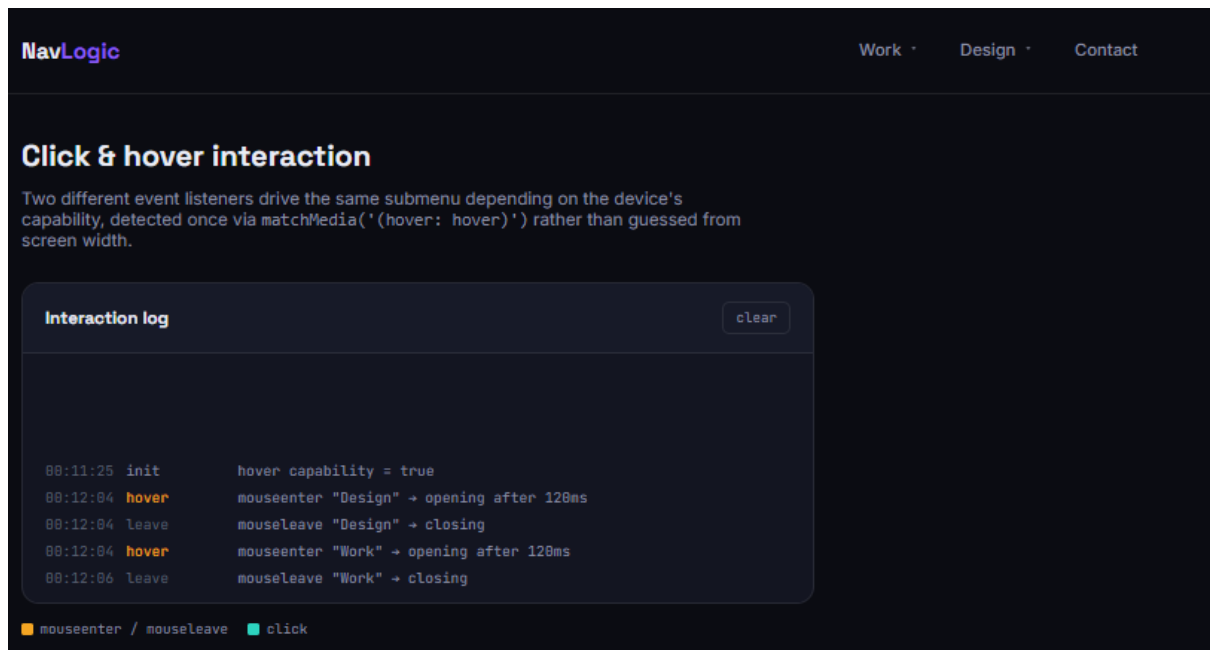
```
Menu toggle — event handling logic
The toggle button owns one piece of state: isOpen. Every listener below either flips
it or reads it — nothing else touches the menu directly.

Event log
● closed clear

00:04:11 click hamburger toggled + open
00:04:13 click link "Contact" selected + closing menu

// the four listeners that make up the toggle logic
toggle.addEventListener('click', e => { e.stopPropagation(); setOpen(!isOpen) })
menu.addEventListener('click', e => { // close after a link is chosen
  if (e.target.tagName === 'A') setOpen(false)
})
document.addEventListener('click', e => { // outside click
  if (isOpen && !menu.contains(e.target)) setOpen(false)
})
document.addEventListener('keydown', e => { // Esc key
  if (e.key === 'Escape' && isOpen) setOpen(false)
})
```

(c) Implement interaction for user actions (click/hover).



(d) Evaluate performance and suggest improvements.

Performance

- Use event listeners instead of inline JavaScript.
- Toggle CSS classes instead of repeatedly changing styles.
- Keep JavaScript lightweight for faster page loading.
- Minimize unnecessary DOM manipulation.

Suggested Improvements

- Add smooth CSS transitions when opening and closing the menu.
- Support keyboard navigation using the keydown event.
- Add ARIA attributes (e.g., `aria-expanded`, `aria-controls`) for accessibility.
- Close the menu automatically when a user clicks outside it.
- Animate the hamburger icon into a close icon for a better user experience.
- Ensure touch-friendly button sizes for mobile devices.

Code of Conduct:

I Nisha K (192411231) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nisha K

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation